

Conteúdo

1	Annotation	1
1.1	Annotation	1
2	Basic	
2.1	Ternary search	1
3	Graphs	
3.1	DSU	1
3.2	Dijkstra	1
3.3	Floyd-Warshall	2
3.4	Bellman-Ford	2
3.5	Kruskal	2
3.6	Bipartite graph check	2
3.7	Topological sort	2
4	DP	
4.1	DP-TP	3
4.2	Knapsack	3
4.3	LIS	3
4.4	LCS	3
4.5	Subset-sum	3
5	Range Queries	
5.1	Merge sort	4
5.2	Segment Tree	4
5.3	Fenwick Tree	4
5.4	Count inversions (Fenwick + compression)	5
6	Strings	
6.1	Hash	5
6.2	KMP	5
6.3	Manacher's algorithm	5
6.4	Trie	6
7	Math	
7.1	Divisors	6
7.2	Modular Combinations	6
7.3	Kadane's algorithm	6
7.4	Sieve of Eratosthenes	7
7.5	Totient function	7
7.6	Matrix operations	7
7.7	Math utilities (gcd, lcm, power, etc.)	8
8	Examples	
8.1	Nodes	8

1 Annotation

1.1 Annotation

```
g++ "$1.cpp" -o "$1" -Wall -Wextra -O2 -Wshadow -fsanitize=address,undefined
g++ "$1.cpp" -o "$1" -Wall -Wextra -Wshadow -O1 -g -fno-omit-frame-pointer
-fsanitize=address,undefined
```

2 Basic

2.1 Ternary search

Ternary Search - Busca ternaria em funcao unimodal (maximo ou minimo)
Complexidade: $O(\log(n))$ - n : intervalo de busca

```
1 // Ternary search on a unimodal function in [l, r].
1 double ternary_search_max(double l, double r, function<double(double)> f) {
1 // while (r - l > 1e-9)
1 for (int it = 0; it < 200; ++it) {
1 double m1 = l + (r - l) / 3.0;
1 double m2 = r - (r - l) / 3.0;
2
2 if (f(m1) < f(m2)) l = m1; // change to > for minimum
2 else r = m2;
2 }
2 return f((l + r) / 2.0);
2 }
```

3 Graphs

3.1 DSU

```
4 struct DSU {
4 vector<int> parent, size;
4 DSU(int n) : parent(n+1), size(n+1) {
4 for(int i=0; i<=n; i++) {parent[i] = i; size[i] = 1;}
5 }
5
5 int find(int u) {
5 if (parent[u] == u) return u;
6
6 return parent[u] = find(parent[u]);
6 }
6
6 void join(int u, int v) {
6 u = find(u); v = find(v);
6 if (u == v) return;
7
7 if (size[u] < size[v]) swap(u, v);
7 parent[v] = u;
7 size[u] += size[v];
8 }
8 };
8
```

3.2 Dijkstra

```
vector<pii> g[MAXN];

void dijkstra(int s, int n, vector<ll>& d) {
    d.assign(n, INF);

    priority_queue<pii, vector<pii>, greater<pii>> q;
    d[s] = 0; q.push({0, s});

    while (!q.empty()) {
        auto [d_u, u] = q.top(); q.pop();
        if (d_u > d[u]) continue;
```

```

    for (auto [v, w] : g[u]) {
        if (d[u] + w < d[v]) {
            d[v] = d[u] + w;
            q.push({d[v], v});
        }
    }
}
}
}

```

3.3 Floyd-Warshall

```

bool floyd_warshall(int n, vector<vector<pii>>& g) {
    vector<vector<ll>> d(n);
    d.assign(n, vector<ll>(n, INF));
    rep(i, n) d[i][i] = 0;
    rep(i, n) for (auto [j, w] : g[i]) d[i][j] = w;

    rep(k, n)
    rep(i, n)
    rep(j, n)
        if (d[i][k] < INF && d[k][j] < INF)
            d[i][j] = min(d[i][j], d[i][k] + d[k][j]);

    rep(i, n)
        if (d[i][i] < 0) return true; // cycle

    return false;
}

```

3.4 Bellman-Ford

```

vector<tii> edges;

void bellman_ford(int s, int n, vector<ll>& dist) {
    dist.assign(n, INF);
    dist[s] = 0;
    vector<int> p(n, -1);

    int x = -1; // cycle detect
    for (int i = 0; i < n; i++) {
        x = -1;
        for (int j = 0; j < edges.size(); j++) {
            auto [u, v, w] = edges[j];
            if (dist[u] != INF && dist[v] > dist[u] + w) {
                dist[v] = dist[u] + w;
                x = v; p[v] = u;
            }
        }
    }

    if (x == -1) {} // no cycle
    else {
        int y = x;
        for (int i = 0; i < n; i++)
            y = p[y];

        vector<int> cycle;
        for (int cur = y; cur = p[cur]) {
            cycle.push_back(cur);
            if (cur == y && cycle.size() > 1) break;
        }
        reverse(cycle.begin(), cycle.end()); // negative cycle
    }
}

```

3.5 Kruskal

```

struct edge {int u, v, w;};
bool comp(edge a, edge b) {return a.w < b.w;};

vector<edge> edges;

int kruskal(int n) {
    DSU ds(n);
    sort(edges.begin(), edges.end(), comp);

    int cost;
    for (auto [u, v, w] : edges) {
        if (ds.find(u) != ds.find(v)) {
            cost += w;
            ds.join(u, v);
        }
    }

    return cost;
}

```

3.6 Bipartite graph check

```

bool bipartite(int n, vector<vector<int>>& g) {
    vector<int> color(n + 1, -1);
    bool ok = true;

    for (int i = 1; i <= n && ok; i++) {
        if (color[i] != -1) continue;
        queue<int> q;
        q.push(i);
        color[i] = 0;

        while (!q.empty() && ok) {
            int u = q.front();
            q.pop();
            for (int v : g[u]) {
                if (color[v] == -1) {
                    color[v] = color[u] ^ 1;
                    q.push(v);
                } else if (color[v] == color[u]) {
                    ok = false;
                    break;
                }
            }
        }
    }

    return ok;
}

```

3.7 Topological sort

Topological Sort (Kahn's Algorithm)
 - $O(V + E)$
 - Requisitos: DAG

```

bool toposort(int n, vector<vector<int>>& g, vector<int>& inDegree) {
    queue<int> q;
    for (int i = 0; i < n; i++) {
        if (inDegree[i] == 0) q.push(i);
    }

    vector<int> order;
    while (!q.empty()) {

```

```

    int u = q.front();
    q.pop();
    order.push_back(u);
    for (int v : g[u]) {
        if (--inDegree[v] == 0) q.push(v);
    }
}

if ((int)order.size() != n) {
    cout << "IMPOSSIBLE\n";
} else {
    for (int i = 0; i < n; i++) {
        cout << order[i] << (i + 1 == n ? '\n' : ' ');
    }
}
return 0;
}
}

```

4 DP

4.1 DP-TP

DP Top-Down (Memoization)

```

const ll INF = 1e18;

const int MAX_STATE_1 = 1005;
const int MAX_STATE_2 = 1005;

const int BASE_VALUE = 0;

ll memo[MAX_STATE_1][MAX_STATE_2];

ll solve(int state1, int state2 /*, const vector<int>& arr */) {
    // 2. Base case: Invalid states (Out of bounds, insufficient capacity, etc.)
    // Return INF for minimization problems, -INF for maximization problems
    if (state1 < 0 || state2 < 0 /*...*/) return INF;

    // 3. Base case: Target state reached
    if (state1 == 0 && state2 == 0 /*...*/) return BASE_VALUE;

    // 4. Memoization check using reference
    ll &ans = memo[state1][state2] /*...*/;

    // Check against the 'uncomputed' flag (-1), NOT the 'invalid path' value
    if (ans != -1) return ans;

    // 5. Recursive case - try all possible transitions
    ans = INF; // Initialize to worst possible outcome (INF for min, -INF for max)

    // for (all possible transitions) {
    //     ll cost_of_transition = ...;
    //     ans = f(ans, solve(new_state1, new_state2) + cost_of_transition);
    // }
    // f is a function that combines the results (e.g., min or max)

    return ans;
}

// memset(memo, -1, sizeof(memo));

```

4.2 Knapsack

```

vector<int> w, v;
// W = knaps capacity, n = n items
// w[i] = weight, v[i] = value
int knapsack(int n, int W) {
    vector<int> dp(W + 1, 0);

    for (int i = 0; i < n; i++) {
        for (int k = W - w[i]; k >= 0; k--) {
            dp[k + w[i]] = max(dp[k + w[i]], dp[k] + v[i]);
        }
    }

    return dp[W];
}

```

4.3 LIS

```

int lis(vector<int>& nums) {
    vector<int> v;

    for (auto x: nums) {
        auto it = lower_bound(v.begin(), v.end(), x);
        // lower -> a > b
        // upper -> a >= b (allow duplicates)

        if (it == v.end()) v.push_back(x);
        else *it = x;
    }

    return v.size();
}

```

4.4 LCS

```

int lcs(string a, string b) {
    vector<vector<int>> m(a.size()+1, vector<int>(b.size()+1));
    for (int i = a.size(); i >= 0; i--) {
        for (int j = b.size(); j >= 0; j--) {
            if (i == a.size() || j == b.size()) {
                m[i][j] = 0; continue;
            }

            if (a[i] == b[j]) m[i][j] = 1 + m[i+1][j+1];
            else m[i][j] = max(m[i][j+1], m[i+1][j]);
        }
    }

    return m[0][0]; // result
}

```

4.5 Subset-sum

```

vector<int> w, v;
vector<vector<int>> memo;

// max sum(v) with sum(w) <= remW
int subsum(int id, int remW) {
    if ((id == w.size()) || (remW == 0)) return 0;
    int &ans = memo[id][remW];
    if (ans != -1) return ans;
    if (w[id] > remW) return ans = subsum(id+1, remW);
}

```

```

    return ans = max(subsum(id + 1, remW),
                     v[id]+subsum(id+1, remW-w[id]));
}

// which values can be created from a list a (int)
int subsumi(int n, int max_v, vector<int> a) {
    vector<int> m(max_v + 10, 0);

    m[0] = 1;
    for (int i = 0; i < n; i++) {
        for (int j = max_v; j >= a[i]; j--) {
            m[j] |= m[j - a[i]];
        }
    }

    return m[max_v];
}

```

5 Range Queries

5.1 Merge sort

```

void merge_sort(vector<int>& a, int l, int r) {
    if (l >= r) return;

    int mid = (l + r) / 2;
    merge_sort(a, l, mid);
    merge_sort(a, mid + 1, r);

    vector<int> temp;
    temp.reserve(r - l + 1);

    int i = l, j = mid + 1;
    while (i <= mid && j <= r) {
        if (a[i] <= a[j]) temp.push_back(a[i++]);
        else temp.push_back(a[j++]);
    }

    while (i <= mid) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);

    for (int k = 0; k < (int)temp.size(); k++) {
        a[l + k] = temp[k];
    }
}

```

5.2 Segment Tree

Segment Tree (Point Update, Range Query)

- Complexidade: $O(\log N)$ para update e query.
- Memoria: $O(4*N)$
- Requisitos:
 - NODE deve ter: static merge(L, R), apply(V), e construtor identidade.

Creditos: Por lua (Lua Guimaraes)
 Fonte: github.com/src-lua/lgf-cpLib

```
#pragma once
```

```

template<typename NODE>
struct SegTree {

```

```

    int N;
    vector<NODE> seg;

    SegTree(int n) : N(n), seg(4 * n) {}

    SegTree(const vector<int>& v) : N(v.size()), seg(4 * v.size()) {
        build(1, 0, N - 1, v);
    }

    void build(int no, int l, int r, const vector<int>& v) {
        if (l == r) {
            seg[no] = NODE(v[l]);
            return;
        }
        int m = (l + r) >> 1;
        build(no << 1, l, m, v);
        build((no << 1) | 1, m + 1, r, v);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    void update(int no, int l, int r, int idx, int val) {
        if (l == r) {
            seg[no].apply(val);
            return;
        }
        int m = (l + r) >> 1;
        if (idx <= m) update(no << 1, l, m, idx, val);
        else update((no << 1) | 1, m + 1, r, idx, val);
        seg[no] = NODE::merge(seg[no << 1], seg[(no << 1) | 1]);
    }

    NODE query(int no, int l, int r, int a, int b) {
        if (b < l || r < a) return NODE();
        if (a <= l && r <= b) return seg[no];
        int m = (l + r) >> 1;
        return NODE::merge(query(no << 1, l, m, a, b),
                           query((no << 1) | 1, m + 1, r, a, b));
    }

    void update(int idx, int val) { update(1, 0, N - 1, idx, val); }
    NODE query(int l, int r) { return query(1, 0, N - 1, l, r); }
};

```

5.3 Fenwick Tree

```

struct Fenwick {
    int n;
    vector<ll> bit;

    Fenwick() {}
    Fenwick(int n): n(n), bit(n + 1, 0) {}

    // add value to index idx (1-indexed)
    void add(int idx, ll val) {
        for (; idx <= n; idx += idx & -idx) {
            bit[idx] += val;
        }
    }

    // sum of [1..idx]
    ll pref_sum(int idx) const {
        ll ans = 0;
        for (; idx > 0; idx -= idx & -idx) {
            ans += bit[idx];
        }
        return ans;
    }
};

```

```

}

ll range_sum(int l, int r) const {
    if (l > r) return 0;
    return pref_sum(r) - pref_sum(l - 1);
}
};

```

5.4 Count inversions (Fenwick + compression)

```

ll count_inversions_fenwick(vector<int> a) {
    if (a.empty()) return 0;

    vector<int> comp = a;
    sort(comp.begin(), comp.end());
    comp.erase(unique(comp.begin(), comp.end()), comp.end());

    int m = (int)comp.size();
    Fenwick fw(m);

    ll inv = 0;
    for (int i = (int)a.size() - 1; i >= 0; i--) {
        int x = (int)(lower_bound(comp.begin(), comp.end(), a[i]) - comp.begin()) + 1;
        inv += fw.pref_sum(x - 1);
        fw.add(x, 1);
    }

    return inv;
}

```

6 Strings

6.1 Hash

String Hash
 precalc() -> O(N)
 StringHash() -> O(|S|)
 gethash() -> O(1)

StringHash hash(s); -> Cria uma struct de StringHash para a string s
 hash.gethash(l, r); -> Retorna o hash do intervalo L R da string (0-Indexado)

IMPORTANT! Chamar precalc() no inicio do codigo

const ll MOD = 131'807'699; -> Big Prime Number
 const ll base = 127; -> Random number larger than the Alphabet

```
const int MAXN = 1e6 + 5;
```

```
const ll MOD = 1e9 + 7; //WA? Muda o MOD e a base
const ll base = 153;
```

```
ll expb[MAXN];
```

```
void precalc(){
    expb[0] = 1;
    for(int i=1; i<MAXN; i++)
        expb[i] = (expb[i-1]*base)%MOD;
}

```

```
struct StringHash{
    vector<ll> hsh;
```

```

StringHash(string &s){
    hsh.assign(s.size()+1, 0);
    for(int i=0; i<s.size(); i++)
        hsh[i+1] = (hsh[i] * base % MOD + s[i]) % MOD;
}

ll gethash(int l, int r){
    return (MOD + hsh[r+1] - hsh[l]*expb[r-l+1] % MOD) % MOD;
}
};

```

6.2 KMP

KMP - Find all occurrences of a pattern string inside a text string

matching(s, t) retorna os indices das ocorrencias de s em t
 autKMP constrói o automato do KMP

Complexidades:
 pi - O(n)
 match - O(n + m)
 construir o automato - O(|sigma|*n)
 n = |padrao| e m = |texto|

```

template<typename T> vector<int> pi(T s) {
    vector<int> p(s.size());
    for (int i = 1, j = 0; i < s.size(); i++) {
        while (j and s[j] != s[i]) j = p[j-1];
        if (s[j] == s[i]) j++;
        p[i] = j;
    }
    return p;
}

```

```

template<typename T> vector<int> matching(T& s, T& t) {
    vector<int> p = pi(s), match;
    for (int i = 0, j = 0; i < t.size(); i++) {
        while (j and s[j] != t[i]) j = p[j-1];
        if (s[j] == t[i]) j++;
        if (j == s.size()) match.push_back(i-j+1), j = p[j-1];
    }
    return match;
}

```

```

struct KMPaut : vector<vector<int>> {
    KMPaut(){}
    KMPaut (string& s) : vector<vector<int>>(26, vector<int>(s.size()+1)) {
        vector<int> p = pi(s);
        auto& aut = *this;
        aut[s[0]-'a'][0] = 1;
        for (char c = 0; c < 26; c++)
            for (int i = 1; i <= s.size(); i++)
                aut[c][i] = s[i]-'a' == c ? i+1 : aut[c][p[i-1]];
    }
};

```

6.3 Manacher's algorithm

Manacher Algorithm
Find every palindrome in string
Complexidade: $O(N)$

```
vector<int> manacher(string &st){
    string s = "$_";
    for(char c : st){ s += c; s += "_"; }
    s += "#";

    int n = s.size()-2;

    vector<int> p(n+2, 0);
    int l=1, r=1;

    for(int i=1, j; i<=n; i++)
    {
        p[i] = max(0, min(r-i, p[l+r-i])) ; //atualizo o valor atual para o valor do palindromo
        espelho na string ou para o total que esta contido

        while( s[i-p[i]] == s[i+p[i]] ) p[i]++;

        if( i+p[i] > r ) l = i-p[i], r = i+p[i];
    }

    for(auto &x : p) x--; //o valor de p[i] e igual ao tamanho do palindromo + 1

    return p;
}
```

6.4 Trie

Trie - Arvore de Prefixos
insert(P) - $O(|P|)$
count(P) - $O(|P|)$
MAXS - Soma do tamanho de todas as Strings
sigma - Tamanho do alfabeto

```
const int MAXS = 1e5 + 10;
const int sigma = 26;

int trie[MAXS][sigma], terminal[MAXS], z = 1;

void insert(string &p){
    int cur = 0;

    for(int i=0; i<p.size(); i++){
        int id = p[i] - 'a';

        if(trie[cur][id] == -1 ){
            memset(trie[z], -1, sizeof trie[z]);
            trie[cur][id] = z++;
        }

        cur = trie[cur][id];
    }

    terminal[cur]++;
}

int count(string &p){
    int cur = 0;
```

```
for(int i=0; i<p.size(); i++){
    int id = (p[i] - 'a');

    if(trie[cur][id] == -1) return 0;

    cur = trie[cur][id];
}
return terminal[cur];
}

void init(){
    memset(trie[0], -1, sizeof trie[0]);
    z = 1;
}
```

7 Math

7.1 Divisors

get_divisors(n) -- $O(\sqrt{N})$

```
vector<int> get_divisors(int n ){
    vector<int> divisors;

    for(int i = 1; i+i <= n; i++){
        if(n % i == 0){
            divisors.push_back(i);
            if(i != n/i) divisors.push_back(n/i);
        }
    }
    return divisors;
}
```

7.2 Modular Combinations

Combinacao modular
Inverso modular
Exponenciacao rapida ($O(\log P)$) - p: potencia
 $O(N)$ fatorial

```
#include <iostream>
#include <vector>

const int MOD = 1e9 + 7;
const int MAX = 2e6 + 5;

ll fact[MAX];
ll invFact[MAX];

// Binary exponentiation for modular inverse
ll modPow(ll base, ll exp) {
    ll res = 1;
    base %= MOD;
    while (exp > 0) {
        if (exp % 2 == 1) res = (res * base) % MOD;
        base = (base * base) % MOD;
        exp /= 2;
    }
    return res;
}
```

```

}

// Fermat's Little Theorem for modular inverse
ll modInverse(ll n) {
    return modPow(n, MOD - 2);
}

// Precompute factorials and inverse factorials
void precompute() {
    fact[0] = 1;
    invFact[0] = 1;
    for (int i = 1; i < MAX; i++) {
        fact[i] = (fact[i - 1] * i) % MOD;
    }
    invFact[MAX - 1] = modInverse(fact[MAX - 1]);
    for (int i = MAX - 2; i >= 1; i--) {
        invFact[i] = (invFact[i + 1] * (i + 1)) % MOD;
    }
}

// O(1) combinations: nCr % MOD
ll nCr(int n, int r) {
    if (r < 0 || r > n) return 0;
    ll num = fact[n];
    ll den = (invFact[r] * invFact[n - r]) % MOD;
    return (num * den) % MOD;
}

```

7.3 Kadane's algorithm

Algoritmo de Kadane
Consegue o max subarray sum
 $O(N)$

```

ll kadane(vector<ll> &arr) {
    ll ans = arr[0];
    ll maxEnding = arr[0];

    for (int i = 1; i < arr.size(); i++) {
        maxEnding = max(maxEnding + arr[i], arr[i]);

        ans = max(ans, maxEnding);
    }

    return ans;
}

```

7.4 Sieve of Eratosthenes

Sieve of Eratosthenes - Encontra o maior divisor primo
Fact -> Fatora um numero <= limite, sai ordenada
Crivo calcula a lista de primos

Crivo_mobius
- 1 se $n=1$
- 0 se n tem algum fator primo ao quadrado
- $(-1)^k$ se n e produto de k primos distintos

A funcao fact adiciona o numero 1 se vc tentar fatorar o 1.
Complexidade:
crivo - $O(n \log(\log N))$

fact - $O(\log(n))$

```

#define MAX 1000
int divi[MAX];
vector<int> primes;

void crivo(int lim) {
    divi[1] = 1;
    for (int i = 2; i <= lim; i++) {
        if (divi[i] == 0) divi[i] = 1, primes.push_back(i);
        for (int j : primes) {
            if (j > divi[i] or i*j > lim) break;
            divi[i*j] = j;
        }
    }
}

void crivo_mobius(ll lim) {
    mobius[1] = 1;
    for (int i = 2; i <= lim; i++) {
        if (!divi[i]) {
            divi[i] = i;
            primes.push_back(i);
            mobius[i] = -1; // primo
        }
        for (int p : primes) {
            if (p > divi[i] || 1LL * i * p > lim) break;
            divi[i*p] = p;
            if (i % p == 0) {
                mobius[i*p] = 0; // quadrado
                break;
            } else {
                mobius[i*p] = -mobius[i];
            }
        }
    }
}

void fact(vector<int>& v, int n) {
    if (n != divi[n]) fact(v, n/divi[n]);
    v.push_back(divi[n]);
}

```

7.5 Totient function

Totiente de Euler - Conta quantos numeros de 1 ate n sao coprimos de n

Complexidade:
 $O(\sqrt{n})$

```

// Totiente
//
// O(sqrt(n))

int tot(int n) {
    int ret = n;

    for (int i = 2; i*i <= n; i++) if (n % i == 0) {
        while (n % i == 0) n /= i;
        ret -= ret / i;
    }
    if (n > 1) ret -= ret / n;

    return ret;
}

```

7.6 Matrix operations

```
const long long MOD = 1e9 + 7;

struct Matrix {
    int n;
    vector<vector<long long>> mat;

    Matrix(int size) {
        n = size;
        mat.assign(n, vector<long long>(n, 0));
    }

    void makeIdentity() {
        for (int i = 0; i < n; i++) {
            mat[i][i] = 1;
        }
    }

    Matrix operator*(const Matrix &other) const {
        Matrix res(n);

        for (int i = 0; i < n; i++) {
            for (int k = 0; k < n; k++) {
                if (mat[i][k] == 0) continue;

                for (int j = 0; j < n; j++) {
                    res.mat[i][j] = (res.mat[i][j] + mat[i][k] * other.mat[k][j]) % MOD;
                }
            }
        }
        return res;
    }
};
```

7.7 Math utilities (gcd, lcm, power, etc.)

```
ll gcd_ll(ll a, ll b) {
    while (b != 0) {
        ll t = a % b;
        a = b;
        b = t;
    }
    return a;
}

ll lcm_ll(ll a, ll b) {
    return a / gcd_ll(a, b) * b;
}

// Bit manipulation utilities
ll count_set_bits(ll n) {
    return __popcount<ll>(n); // __builtin_popcount(n);
}
```

8.1 Nodes

Colecao de Nodes para Segment Tree / Sparse Table
 Copie o Node que voce precisa para o seu codigo.
 Requisitos: Cada Node deve ter static merge() e construtor identidade.
 Para Lazy: Node deve ter apply(TAG, l, r).

Creditos: Por lua (Lua Guimaraes)
 Fonte: github.com/src-lua/lgf-cpLib

// NODES BASICOS (Idempotentes - para Sparse Table tambem)

```
#pragma once
using ll = long long int;
```

```
struct SumNode {
    ll val = 0;
    SumNode(ll v = 0) : val(v) {}

    static inline SumNode merge(const SumNode& l, const SumNode& r) {
        return SumNode(l.val + r.val);
    }
};
```

```
// Para Lazy com AddTag
void apply(ll add, int len) {
    val += add * len;
}
```

```
struct MinNode {
    ll val = LLONG_MAX;
    int pos = -1;
    MinNode(ll v = LLONG_MAX, int p = -1) : val(v), pos(p) {}

    static inline MinNode merge(const MinNode& l, const MinNode& r) {
        return l.val <= r.val ? l : r;
    }
};
```

// NODES AVANCADOS

// --- Matrix 2x2 Node (para Fibonacci, recorrências lineares) ---
 // Exemplo: fib(n) = [[1,1],[1,0]]^n * [[1],[0]]
 const ll MOD = 1e9 + 7;

```
struct Matrix2x2Node {
    ll a[2][2];

    Matrix2x2Node() {
        memset(a, 0, sizeof a);
        a[0][0] = a[1][1] = 1; // identidade
    }

    Matrix2x2Node(ll a00, ll a01, ll a10, ll a11) {
        a[0][0] = a00; a[0][1] = a01;
        a[1][0] = a10; a[1][1] = a11;
    }

    static inline Matrix2x2Node merge(const Matrix2x2Node& l,
                                       const Matrix2x2Node& r) {
        Matrix2x2Node res;
        for(int i = 0; i < 2; i++)
            for(int j = 0; j < 2; j++) {
                res.a[i][j] = 0;
                for(int k = 0; k < 2; k++)
                    res.a[i][j] = (res.a[i][j] + l.a[i][k] * r.a[k][j]) % MOD;
            }
        return res;
    }
};
```

```
};

// --- Range Sum com contador ---
// Util para queries tipo: "quantos elementos < x no range [l,r]"
struct CountNode {
    ll sum = 0;
    int cnt = 0;
    CountNode(ll s = 0, int c = 0) : sum(s), cnt(c) {}

    static inline CountNode merge(const CountNode& l, const CountNode& r) {
        return CountNode(l.sum + r.sum, l.cnt + r.cnt);
    }
};

// --- Max Subarray Sum Node ---
// Mantem soma total, melhor prefixo, melhor sufixo e melhor subarray no range.
struct SubarraySumNode {
    ll sum = 0;
    ll pref = LLONG_MIN / 4;
    ll suff = LLONG_MIN / 4;
    ll best = LLONG_MIN / 4;

    // Identidade para query fora do range.
    SubarraySumNode() = default;

    // Folha com valor unico.
    SubarraySumNode(ll v) : sum(v), pref(v), suff(v), best(v) {}

    static inline SubarraySumNode merge(const SubarraySumNode& l,
                                         const SubarraySumNode& r) {
        SubarraySumNode ans;
        ans.sum = l.sum + r.sum;
        ans.pref = max(l.pref, l.sum + r.pref);
        ans.suff = max(r.suff, r.sum + l.suff);
        ans.best = max({l.best, r.best, l.suff + r.pref});
        return ans;
    }
};

// --- Funcao Linear Node (composicao de funcoes: f(x) = ax + b) ---
// Muito util para composicao de funcoes em arvores
struct LinearFunctionNode {
    ll a = 1, b = 0; // f(x) = ax + b

    LinearFunctionNode(ll _a = 1, ll _b = 0) : a(_a % MOD), b(_b % MOD) {}

    // Composicao: (f o g)(x) = f(g(x)) = f(gx + h) = a(gx + h) + b
    // = (ag)x + (ah + b)
    static inline LinearFunctionNode merge(const LinearFunctionNode& f,
                                           const LinearFunctionNode& g) {
        return LinearFunctionNode(
            (f.a * g.a) % MOD,
            (f.a * g.b + f.b) % MOD
        );
    }

    // Aplica a funcao: f(x) = ax + b
    ll eval(ll x) const {
        return (a * x + b) % MOD;
    }
};

// EXEMPLOS DE USO

/*
// --- Segment Tree com SumNode ---
SegTree<SumNode> st(n);
st.update(i, SumNode(val));
SumNode result = st.query(l, r);
cout << result.val << '\n';
*/
```

```
// --- Sparse Table com MinNode (RMQ) ---
SparseTable<MinNode> sp(v);
MinNode minimo = sp.query(l, r);
cout << "Min: " << minimo.val << " at position " << minimo.pos << '\n';

// --- Matrix para Fibonacci ---
SegTree<Matrix2x2Node> st(n);
Matrix2x2Node fib(1, 1, 1, 0);
st.update(i, fib);

// --- Composicao de Funcoes Lineares ---
SegTree<LinearFunctionNode> st(n);
st.update(i, LinearFunctionNode(2, 3)); //  $f(x) = 2x + 3$ 
st.update(j, LinearFunctionNode(4, 1)); //  $g(x) = 4x + 1$ 
//  $\text{merge}(f, g) = f(g(x)) = 2(4x + 1) + 3 = 8x + 5$ 
LinearFunctionNode composed = st.query(l, r);
ll result = composed.eval(x);
*/
```
